

系统分析师考前几页纸

1、系统工程方法

系统工程方法	关键点
霍尔三维结构 “硬科学”方法论	逻辑维：逻辑维即解决问题的逻辑过程。 时间维：时间维即是工作进程。 知识维：知识维即是专业科学知识。 应用场景：组织和管理大型工程建设项目
切克兰德方法 “软科学”方法论	核心不是“最优化”，而是“比较”和“探寻” 5 步骤：问题现状说明、理清问题的关联因素、建立概念模型、比较和实施。

2、线性规划

(1) 线性规划的一些特点：

线性规划的可行解区是由一组线性约束条件形成的，从几何意义来说，就是由一些线性解面围割形成的区域，不一定是封闭的多边形或多面体。

如果存在两个最优解，则连接这两点的线段内所有的点都是最优解，而线段两端延长线上可能会超出可行解区。

增加一个约束条件时，要么缩小可行解区（新的约束条件分割了原来的可行解区），要么可行解区不变（新的约束条件与原来的可行解区不相交）。

如果最优解在可行解区边界某个非顶点处达到，则随着等值域向某个方向移动，目标函数的值会增加或减少（与最优解矛盾）或没有变化（在此段边界上都达到最优解），从而仍会在可行解区的某个顶点处达到最优解。若最优解存在且唯一，则可以从可行解区顶点处比较目标函数值来求解。

(2) 线性规划的标准型（standard form of linear programming）是线性规划模型的标准形式。其主要特征为：

- 目标函数为极大化类型；
- 所有的约束条件都是等式；
- 所有约束方程右端的常数都是非负的；
- 所有决策变量都是非负的

3、死锁

所谓死锁，是指两个以上的进程互相都要求对方已经占有的资源导致无法继续运行下去的现象。

(1) 死锁的四大条件：互斥；保持和等待；不剥夺；环路等待。

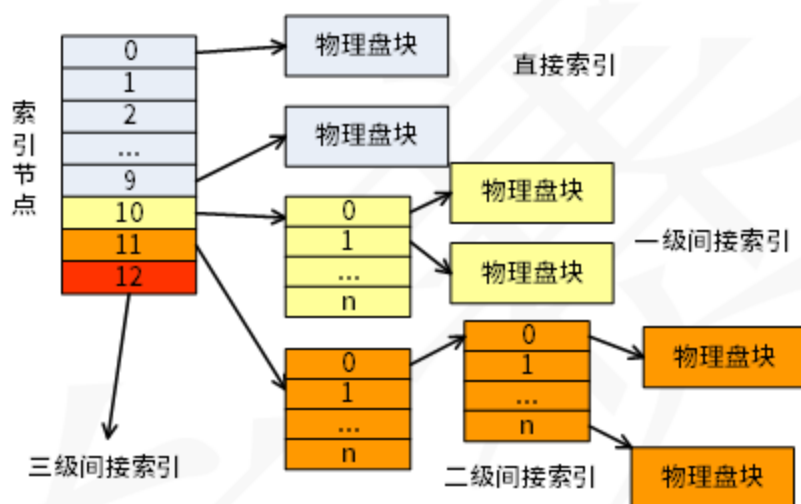
(2) 死锁的预防：打破四大条件。

(3) 死锁的避免：银行家算法。

(4) 死锁资源数计算问题：根据题干给出的进程和资源分配，判断形成死锁的最小资源数或其它参数。

对于这种情况，分配资源时每个进程得到可以完成进程的资源数减一，此时是形成死锁的最差情况，在此情况下多 1 个资源即可解决死锁问题，即不可能形成死锁。假设 m 个进程各自需要 w 个 R 资源，系统中共有 n 个 R 资源，此时不可能形成死锁的条件是： $m \cdot (w-1) + 1 \leq n$

4、索引文件结构



(1) 索引节点对应的索引方式一般题干会给出，没有给出的默认按照如图所示方式理解，下面的文件大小依图给出计算过程。

(2) 根据物理块大小（假设 1KB）和地址项长度（假设 4B），可以计算存放间接索引的物理块可以存放的地址项个数：物理块大小/地址项长度，向下取整（ $1KB/4B=256$ ，注意单位和进制转换）。

(3) 直接索引（即索引节点直接指向实际存储文件的物理块），能够表示的逻辑页号范围是 0-9，能够表示的文件大小是 $10 \times 1KB$ 。【访问 1 次对应磁盘找到数据】

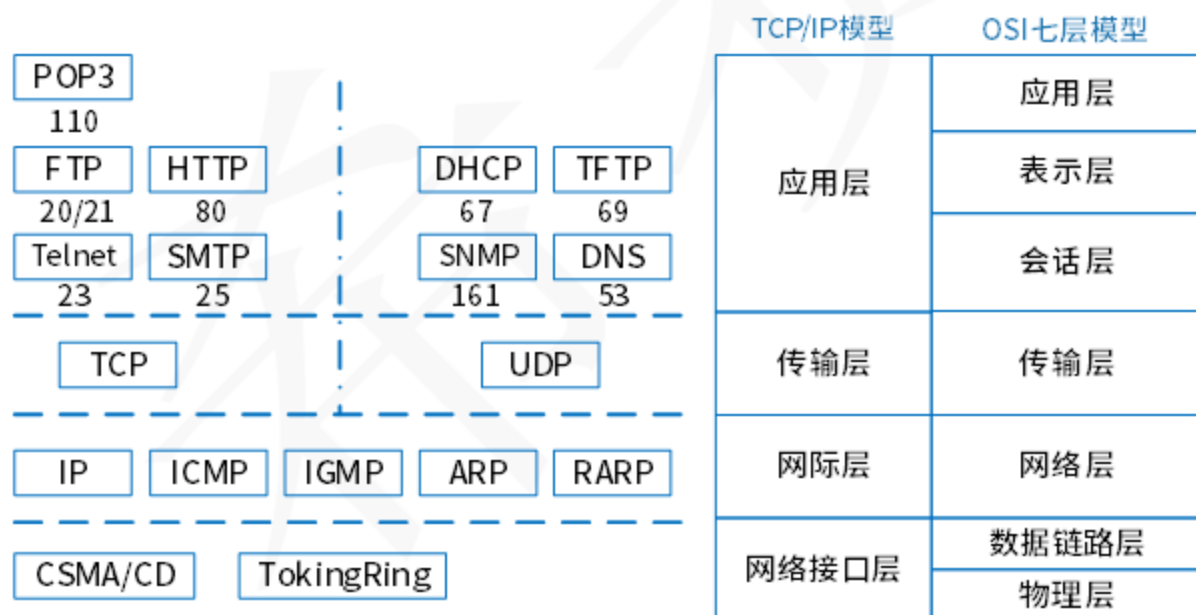
(4) 一级间接索引（即索引节点指向的物理块存放的是一级间接索引表的地址项，共 256 个，可以指向 256 个实际存储文件的物理块），能够表示的逻辑页号范围是 10-265，能够表示的文件大小是 $256 \times 1KB$ 。【访问 2 次对应磁盘找到数据】

(5) 二级间接索引（即索引节点指向的物理块存放的是二级间接索引表的地址项，共 256 个，可以指向 256 个一级间接索引表地址项的物理块，每个物理块指向实际存储文件的地址项有 256 个，最终指向的物理块共有 256×256 个），能够表示的逻辑页号范围是 266-65801，能够表示的文件大小是 $65536KB$ 。【访问 3 次对应磁盘找到数据】

5、OSI/RM 七层协议

层次	名称	主要功能	主要设备及协议
7	应用层	实现具体的应用功能	POP3、FTP、HTTP、Telnet、SMTP DHCP、TFTP、SNMP、DNS
6	表示层	数据的格式与表达、加密、压缩	
5	会话层	建立、管理和终止会话	
4	传输层	端到端的连接	TCP、UDP
3	网络层	分组传输和路由选择	三层交换机、路由器 IP、ICMP、IGMP、ARP、RARP
2	数据链路层	传送以帧为单位的信息	网桥、交换机（多端口网桥）、网卡 PPP、PPTP、L2TP、SLIP
1	物理层	二进制传输	中继器、集线器（多端口中继器）

6、TCP/IP 协议簇



7、局域网

(1) 概念：

局域网是指在有限地理范围内将若干计算机通过传输介质互联成的计算机组（即通信网络），通过网络软件实现计算机之间的文件管理、应用软件共享、打印机共享、工作组内的日程安排、电子邮件和传真通信服务等功能。局域网是封闭型的。

(2) 局域网拓扑结构

局域网专用性非常强，具有比较稳定和规范的拓扑结构。常见的局域网拓扑结构有星状结构、树状结构、总线结构、环形结构和网状结构。

8、关系代数

(1) 并（结果为二者元组之和去除重复行）

(2) 交（结果为二者重复行）

(3) 差（前者去除二者重复行）：以元组行作为整体进行判断，类似于集合运算。

(4) 笛卡尔积：结果列数为二者属性列数之和，行数为二者元组行数的乘积。

两个表做笛卡尔积，结果表的元组由前表与后表的元组拼接而成，不同的排列组合形成不同的结果元组。

(5) 投影（筛选符合条件的属性列）

(6) 选择（筛选符合条件的元组）：属性名可以依次标序号，直接以数字形式出现在表达式中。

(7) 自然连接：结果列数为二者属性列数之和减去重复列，行数为二者同名属性列其值相同的结果元组。笛卡尔积、选择、投影的组合表示可以与自然连接等价。

9、范式

规范化是为了解决数据冗余、删除异常、插入异常、更新异常等问题。

第一范式（1NF）：在关系模式 R 中，当且仅当所有域只包含原子值，即每个属性都是不可再分的数据项，则称关系模式 R 是第一范式。

第二范式（2NF）：当且仅当关系模式 R 是第一范式（1NF），且每一个非主属性完全依赖候选键（没有不完全依赖）时，则称关系模式 R 是第二范式。

第三范式（3NF）：当且仅当关系模式 R 是第二范式（2NF），且 R 中没有非主属性传递依赖于候选键时，则称关系模式 R 是第三范式。

BC 范式（BCNF）：设 R 是一个关系模式，F 是它的依赖集，当且仅当其 F 中每个依赖的决定因素必定包含 R 的某个候选码时，R 属于 BCNF。【当且仅当关系模式 R 是第三范式（3NF）】

注：4NF 是限制关系模式的属性间不允许有非平凡且非函数依赖的多值依赖。。

10、事务的特性（ACID）

原子性（Atomicity）是指事务包含的所有操作要么全部成功，要么全部失败回滚。这些操作是一个整体，不能部分地完成。

一致性（Consistency）是指事务必须使数据库从一个一致性状态变换到另一个一致性状态，也就是说一个事务执行之前和执行之后都必须处于一致性状态。

隔离性（Isolation）是指一个事务的执行不能被其它事务干扰，即一个事务内部的操作及使用的数据对并发的其它事务是隔离的。

持久性（Durability，永久性）是指一个事务一旦被提交了，那么对数据库中的数据的改变就是永久性的，无论发生何种故障，都不应对其有任何影响。

11、NoSql 数据库分类

类型	典型产品	典型应用场景	数据模型	优点	缺点
Key-Value	Memcached、Redis	缓存、日志等	key-value/全键值对	扩展性好，灵活性强，大量操作时性能高	数据无结构化
列式存储	BigTable、Hbase、Cassandra	分布式数据存储和管理	列簇式存储	可扩展性强，查找速度快，复杂度低	功能相对局限，不支持事务强一致性
文档型	MongoDB、CouchDB	Web 应用	key-value，value 为结构化或半结构化数据	结构灵活，可以根据 value 构建索引	查询性能不高、缺乏统一语言
图数据库	Neo4j、OrientDB	社交网络、推荐系统等	图存储三个字段：节点，关系和属性	支持复杂的图形算法	复杂度高，只能支持一定数据规模，实现困难

12、反规范化的技术手段以及优缺点

技术手段	说明
增加派生性冗余列	已有单价和数量列，增加“总价”列
增加冗余列	已有学号列，增加“姓名”列
重新组表	把拆分的表重新组表
分割表	把用户表做水平分割，长沙的用户存在长沙，上海的用户存在上海

反规范化的优点：连接操作少，检索快、统计快；需要查的表减少，检索容易。

反规范化的缺点	解决方案
数据冗余，需要更大存储空间	无解

插入、更新、删除操作开销更大	无解
数据不一致 可能产生添加、修改、删除异常	1、触发器数据同步 2、应用程序数据同步
更新和插入代码更难写	无解

13、信息资源管理（IRM）

（1）国民经济和社会发展三大战略资源：

信息资源：无限的、可再生、可共享【业务信息化基础上形成的各种信息】。

材料与能源：有限的、不可再生的、不可共享的。

（2）IRM 基本内容（三个主题）：资源管理的方向和控制；建立企业信息资源指导委员会；建立信息资源的组织机构。

（3）IRM 目的：通过企业内外信息流的畅通和信息资源的有效利用，提高企业的效益和竞争力。

（4）数据资源管理和信息处理管理

IRM 包括：数据资源管理和信息处理管理。数据资源管理：强调对数据的控制（维护和安全）。信息处理管理：关心企业管理人员如何获取和处理信息（流程和方法），并且强调企业中信息资源的重要性。

（5）数据管理（IRM 的基础）

不同于数据库管理

确定数据规划、数据应用、数据标准、数据内容和数据范围等。

（6）IRM 基础标准：数据元素标准；信息分类编码标准；用户视图标准；概念数据库标准；逻辑数据库标准

（7）IRM 制定和实施的原则

- 不绝对可灵活，允许例外，但不能例外当正规。
- 标准必须从实际出发切实可行，容易执行，不迁就落后，标准一致性。
- 逐步制定而不是一步到位，细节不重要，必须加以宣传。
- 管理部门必须支持且乐于帮助执行。

（8）信息资源的分类

分类维度	描述
管理维度	一般有 2 种情况：一是专门的业务部门所采用的分类体系；二是综合部门从信息资源登记和管理的角度提出的分类。
信息来源维度	比较简单，一般按照信息资源提供部门来设置目录。 优势：从分类信息的赋值角度极大地简化了工作量，无需培训、重复录入，该

	部门初始化后可以复用；使用者不需要学习或了解特定分类体系的内容，查找过程更加简单和直接。
应用主题维度	最复杂。同一个信息资源根据其服务和应用目标不同，会有不同的分类。

(9) IRM 基础标准

①数据元素标准。数据元素是最小的不可再分的信息单位，是一类数据的总称，它是数据对象的抽象。数据元素标准又可分为命名标准、标识标准和一致性标准。研究表明，数据元素具有“原子意义”，根据企业的类型和规模，数据元素不仅在数目上存在统计规律，而且还有比较稳定的对象集。如果不重视数据元素的标准化，即缺少数据元素命名标准，或者未考虑数据元素创建和使用规划，就会导致数据处理系统中所使用的数据名称庞杂混乱。例如，在数据处理系统中的“职工姓名”“员工姓名”“职员姓名”等。

②信息分类编码标准。信息分类就是根据信息内容的属性或特征，将信息按一定的原则和方法进行区分和归类，并建立起一定的分类系统和排列顺序，以便管理和使用信息。信息分类编码标准是信息资源标准中最基础的标准。例如，一种典型的分类方法为 ABC 法。A 类编码对象是指在系统中不单独设立编码表，它们反映了基本表中的信息分类编码对象，是基本表的一个视图。例如，身份证号(国家标准)、设备编码(企业标准)等。B 类编码对象是指在系统中单独设立编码表(基本表)。例如，职称编码(国家标准)、设备配件编码(企业标准)等。C 类编码对象是指在应用系统中有一些码表短小而使用频率很大，把这些对象统一设在一个编码库管理就可以了。例如，人的性别、文化程度等。

③用户视图标准。用户视图是一些数据元素的集合，它反映了最终用户对数据实体的看法。用户视图是数据在系统外部而不是内部的表象，是系统的输入或输出的媒介或手段。用户视图主要包括企业管理的表单、报表、屏幕数据格式等。用户视图的规范化管理包括用户视图名称、标识和组成的管理。规范并简化用户视图是企业内外信息共享和交换的基础。

④概念数据库标准。概念数据库是最终用户对数据存储的看法，是对用户信息需求的综合概括。概念数据库标准包括数据库名称、标识、主关键字和数据内容列表。列表项可以是数据元素，也可以是数据元组。概念数据就是主题数据库的概要信息。企业的概念数据库标准是指列出整个企业所有的主题数据库的概要信息，它是经过 SDP (战略数据规划)工作之后完成的。

⑤逻辑数据库标准。逻辑数据库是系统分析与设计的基础，是对概念数据库的进一步分解和细化，一个逻辑主题数据库由一组规范化的基本表构成。由概念数据库演化为逻辑数据库，主要工作是采用数据结构规范化的理论与方法。

14、信息系统开发方法

结构化开发方法

(1) 自顶向下，逐步分解求精；

- (2) 严格区分阶段，阶段产出标准化；
- (3) 应变能力差。

原型法【需求阶段，作用是展示及获取需求】

- (1) 针对需求不明确；
- (2) 按功能分：水平原型（界面）、垂直原型（复杂算法）；
- (3) 按最终结果分：抛弃式原型、演化式原型。

面向对象方法

- (1) 自底向上；
- (2) 阶段界限不明；
- (3) 更好应变、更好复用；
- (4) 符合人们的思维习惯。

面向服务的方法

- (1) 粗粒度、松耦合；
- (2) 标准化和构件化；
- (3) 面向服务方法的三个抽象级别：

操作：位于最底层，对标函数方法这个层次。

服务：代表操作的逻辑分组。

业务流程：为实现特定业务目标而执行的一组长期运行的动作或活动。业务流程的例子有：接纳新员工、出售产品或服务并完成订单。

15、企业应用集成

	集成点	效果	解题关键点
界面集成	界面层	统一入口，产生“整体”感觉	“整体”感觉 最小代价实现一体化操作
数据集成	数据层	不同来源的数据逻辑或物理上“集中”	其他集成方法的基础
控制集成	应用逻辑层	调用其它系统已有方法，达到集成效果	/
业务流程集成 (过程集成)	应用逻辑层	跨企业，或优化流程而非直接调用	企业之间的信息共享能力
门户集成	/	将内部系统对接到互联网上	发布到互联网上

16、电子政务

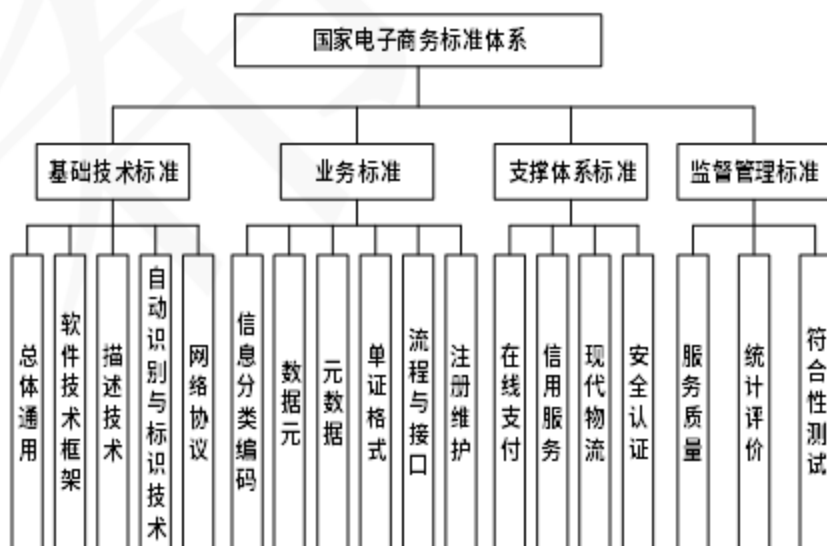
电子政务主要有 3 类角色：政府（Government）、企（事）业单位（Business）及公民（Citizen）。如果有第 4 类就是公务员（Employee）。

类型	应用
G2G	基础信息的采集、处理和利用，如：人口信息、地理信息 各级政府决策支持
G2E	政府内部管理系统
G2B	政府给企业单位颁发【各种营业执照、许可证、合格证、质量认证】
B2G	企业向政府缴税 企业向政府供应各种商品和服务【含竞/投标】 企业向政府提建议，申诉
G2C	社区公安和水、火、天灾等与公共安全有关的信息 户口、各种证件和牌照的管理
C2G	个人应向政府缴纳的各种税款和费用 个人向政府反馈民意【征求群众意见】 报警服务【盗贼、医疗、急救、火警等】

17、电子商务

(1) 电子商务主要有 2 类角色：企业（Business）及个人（Customer）。

(2) 国家电子商务标准体系。



(3) 电子商务的类型：

类型	应用
B2B（企业对企业）	阿里巴巴，慧聪网

C2C（消费者对消费者）	闲鱼
B2C（企业对消费者）	京东，天猫
C2B（消费者对企业）【非主流】	个人给企业提供咨询服务
O2O（Online To Offline）【线上对线下】	团购

18、可行性分类

经济可行性：成本收益分析，包括建设成本、运行成本和项目建设后可能的经济收益。

技术可行性：技术风险分析，现有的技术能否支持系统目标的实现，现有资源（员工、技术积累、构件库、软硬件条件）是否足以支持项目的实施。

法律可行性【社会可行性】：不能与国家法律或政策相抵触。

用户使用可行性：从信息系统用户的角度评估系统的可行性。

管理可行性：系统与现有管理机制的一致性，改革的可能性。

运行可行性：用户方便使用的程度，及用户的素质和培训要求。

19、与逆向工程相关的概念有重构、设计恢复、再工程和正向工程

(1) 重构（restructuring）。重构是指在同一抽象级别上转换系统描述形式。

(2) 设计恢复（design recovery）。设计恢复是指借助工具从已有程序中抽象出有关数据设计、总体结构设计和过程设计等方面的信息。

(3) 逆向工程（reverse engineering）。逆向工程是分析程序，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程是设计的恢复过程。

(4) 正向工程（forward engineering）。正向工程是指不仅从现有系统中恢复设计信息，而且使用该信息去改变或重构现有系统，以改善其整体质量。

(5) 再工程（re-engineering）。再工程是对现有系统的重新开发过程，包括逆向工程、新需求的考虑过程和正向工程三个步骤。

20、需求的层次

业务需求：是指反应企业或客户对系统高层次的目标要求，通常来自项目投资入、购买产品的客户、客户单位的管理人员、市场营销部门或产品策划部门等。通过业务需求可以确定项目视图和范围，为以后的开发工作奠定了基础。

用户需求：描述的是用户的具体目标，或用户要求系统必须能完成的任务。也就是说，用户需求描述了用户能使用系统来做些什么。

系统需求：是从系统的角度来说明软件的需求，包括功能需求、非功能需求和设计约束等。

21、软件开发模型

(1) 原型模型

典型的原型开发方法模型。适用于【需求不明确的场景】，可以帮助用户明确需求。

(2) 瀑布模型

瀑布模型是将软件生存周期中的各个活动规定为以线性顺序连接的若干阶段的模型，包括需求分析、设计、编码、运行与维护。

瀑布模型的特点是容易理解，管理成本低，每个阶段都有对应的成果产物，各个阶段有明显的界限划分和顺序要求，一旦发生错误，整个项目推倒重新开始。

适用于需求明确的项目，一般表述为需求明确或二次开发，或者对于数据处理类型的项目

(3) 增量模型

融合了瀑布模型的基本成分和原型实现的迭代特征，可以有多个可用版本的发布，核心功能往往最先完成，在此基础上，每轮迭代会有新的增量发布，核心功能可以得到充分测试。强调每一个增量均发布一个可操作的产品。

(4) 螺旋模型

典型特点是引入了风险分析。以原型为基础+瀑布模型，最主要的特点在于加入了风险分析。它是由制定计划、风险分析、实施工程、客户评估这一循环组成的，并且从概念项目开始第一个螺旋。

(5) V 模型

测试贯穿于始终，测试分阶段，测试计划提前。

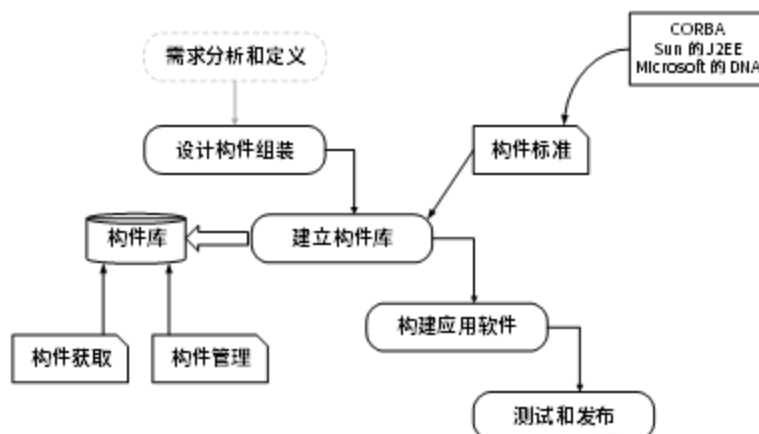
(6) 喷泉模型

典型的面向对象的模型。特点是迭代、无间隙。会将软件开发划分为多个阶段，但各个阶段无明显界限，并且可以迭代交叉。早期著名的面向对象模型。

(7) 快速应用开发 RAD

概念：RAD 是瀑布模型的一个高速变种，适用比传统生命周期快得多的开发方法，它强调极短的开发周期，通常是用基于构件的开发方法获得快速开发。

(8) 构件组装模型



(9) 统一过程（在软考中 UP、RUP 都指统一过程）

典型特点是用例驱动、以架构为中心、迭代和增量。统一过程把一个项目分为四个不同的阶段：

构思阶段（初始阶段）：定义最终产品视图和业务模型；确定系统范围

细化阶段：设计及确定系统架构；制定工作计划及资源要求

构建阶段：开发剩余构件和应用程序功能，把这些构件组装为产品，并进行详细测试

移交阶段（交付阶段）：确保软件对最终用户是可用的，进行β测试，制作产品发布版本

9 个核心 workflow：业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境。

(10) 敏捷开发概念：敏捷开发是一种以人为核心、迭代、循序渐进的开发方法，适用于小团队和小项目，具有小步快跑的思想。常见的敏捷开发方法有极限编程法、水晶法、并列争球法和自适应软件开发方法。

敏捷宣言：

个体和交互胜过过程和工具

可工作的软件胜过大量的文档【实现与测试是核心】

客户合作胜过合同谈判

响应变化胜过遵循计划

敏捷方法：

敏捷方法	特点总结
极限编程 (XP)	在一些对费用控制严格的公司中的使用，非常有效
水晶方法	探索了用最少纪律约束而仍能成功的方法，从而在产出效率与易于运作上达到一种平衡
开放式源码	程序开发人员在地域上分布很广【其它方法强调集中办公】
SCRUM	明确定义了的可重复的方法过程。适用于任何复杂的或创新性的项目
功用驱动开发方法	编程开发人员分成两类：首席程序员和“类”程序员

(FDD)	
ASD 方法	其核心是三个非线性的、重叠的开发阶段：猜测、合作与学习
动态系统开发方法 (DSDM)	倡导以业务为核心

22、需求获取方法

方法	特点
用户访谈	1 对 1-3，有代表性的用户，了解主观想法，交互好。成本高，要有领域知识支撑
问卷调查	用户多，无法一一访谈，成本低
采样 (抽样调查)	基于数理统计，降低成本，快速获取 采样技术包括：随机采样、分层采样、聚类采样等，具体采用哪种方法根据文档数量、样本大小和实际情况决定
情节串联板 【原型前身】	一系列图片，通过这些图片来讲故事。类型包括：被动式、主动式和交互式
联合需求计划	高度组织的群体会议，各方参与，了解想法，消除分歧，交互好，成本高
需求记录技术	需求记录工具有：任务卡片、场景说明、用户故事和 Volere 白卡等

23、内聚与耦合：

内聚：衡量模块内部各元素结合的紧密程度

内聚类型	描 述
功能内聚	完成一个单一功能，各个部分协同工作，缺一不可
顺序内聚	处理元素相关，而且必须顺序执行
通信内聚	所有处理元素集中在一个数据结构的区域上
过程内聚	处理元素相关，而且必须按特定的次序执行
瞬时内聚（时间内聚）	所包含的任务必须在同一时间间隔内执行
逻辑内聚	完成逻辑上相关的一组任务
偶然内聚（巧合内聚）	完成一组没有关系或松散关系的任务

耦合：度量不同模块间互相依赖的程度

耦合类型	描 述
非直接耦合	两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的

数据耦合	一组模块借助参数表传递简单数据
标记耦合	一组模块通过参数表传递记录信息（数据结构）
控制耦合	模块之间传递的信息中包含用于控制模块内部逻辑的信息
外部耦合	一组模块都访问同一全局简单变量，而且不是通过参数表传递该全局变量的信息
公共耦合	多个模块都访问同一个公共数据环境
内容耦合	一个模块直接访问另一个模块的内部数据；一个模块不通过正常入口转到另一个模块的内部；两个模块有一部分程序代码重叠；一个模块有多个入口

24、面向对象设计原则：

单一职责原则：设计目的单一的类

开放-封闭原则：对扩展开放，对修改封闭

李氏（Liskov）替换原则：子类可以替换父类

依赖倒置原则：要依赖于抽象，而不是具体实现；针对接口编程，不要针对实现编程

接口隔离原则：使用多个专门的接口比使用单一的总接口要好

组合重用原则：要尽量使用组合，而不是继承关系达到重用目的

迪米特（Demeter）原则（最少知识法则）：一个对象应当对其他对象有尽可能少的了解

25、实体类、控制类与边界类

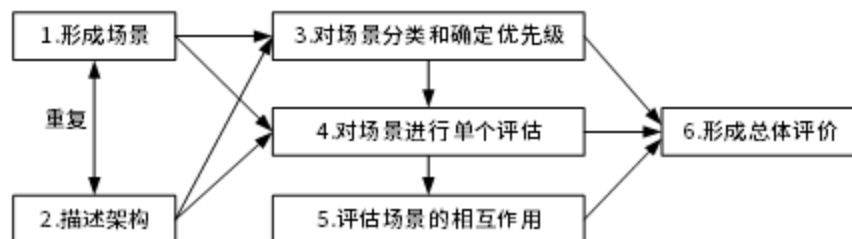
实体类映射需求中的每个实体，实体类保存需要存储在永久存储体中的信息，例如，在线教育平台系统可以提取出学员类和课程类，它们都属于实体类。

控制类是用于控制用例工作的类，一般是由动宾结构的短语（“动词+名词”或“名词+动词”）转化来的名词，例如，用例“身份验证”可以对应于一个控制类“身份验证器”，它提供了与身份验证相关的所有操作。

边界类用于封装在用例内、外流动的信息或数据流。边界类位于系统与外界的交接处，包括所有窗体、报表、打印机和扫描仪等硬件的接口，以及与其他系统的接口。

26、架构评估：SAAM 和 ATAM

（1）SAAM：最初用于分析架构的可修改性，后扩展到其它质量属性。



(2) ATAM: 在 SAAM 的基础上发展起来的。ATAM 希望揭示出架构满足特定质量目标的情况，使架构设计师更清楚地认识到质量目标之间的联系，即如何权衡多个质量目标。

主要针对性能、可用性、安全性和可修改性。

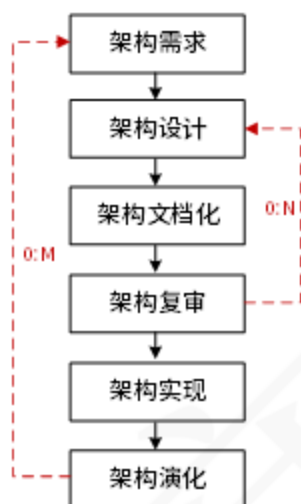


27、ABSD（基于架构的软件开发模型）

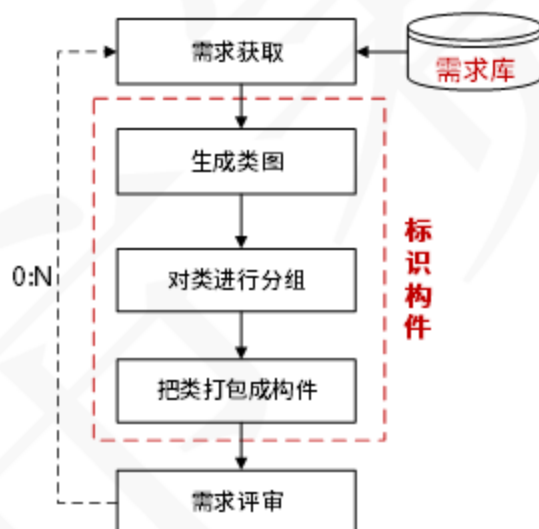
ABSD 方法是架构驱动，即强调由业务【商业】、质量和功能需求的组合驱动架构设计。

ABSD 方法有三个基础。第一个基础是功能的分解。在功能分解中，ABSD 方法使用已有的基于模块的内聚和耦合技术；第二个基础是通过选择架构风格来实现质量和业务需求；第三个基础是软件模板的使用。

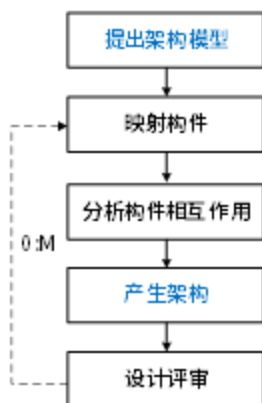
流程：ABSD 能很好的【支持软件重用】。ABSD 方法是一个自顶向下，递归细化的方法。软件系统的体系结构通过该方法得到细化，直到能产生软件构件和类。



(1) 架构需求过程



(2) 架构设计过程



(3) 架构文档化

架构文档化过程的主要输出结果是架构规格说明和测试架构需求的质量设计说明书这两个文档。

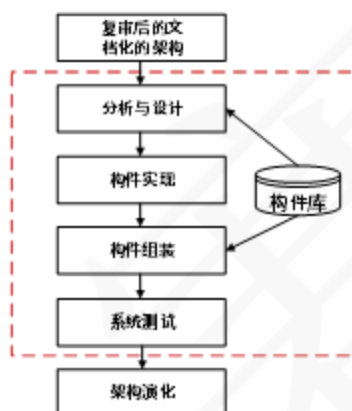
文档的完整性和质量是软件架构成功的关键因素。

关于文档的三大注意事项：文档要从使用者的角度进行编写；必须分发给所有与系统有关的开发人员；必须保证开发者手上的文档是最新的。

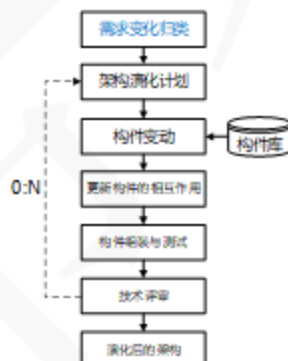
(4) 架构复审

架构复审【架构评估】的目的是标识潜在的风险，及早发现架构设计中的缺陷和错误。

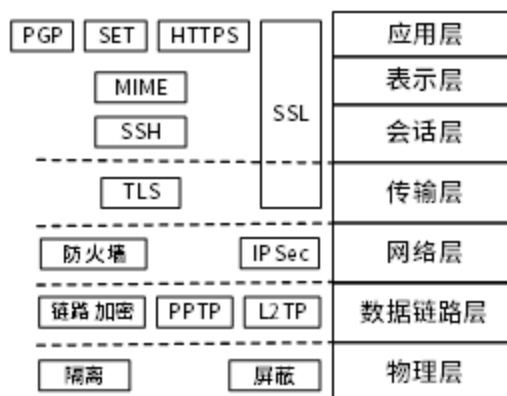
(5) 架构实现



(6) 架构演化



28、安全协议



PGP (Pretty Good Privacy)：针对邮件和文件的混合加密系统。

SSL (Secure Sockets Layer)：工作在传输层至应用层。

TLS (Transport Layer Security)：传输层安全协议。

SET (Secure Electronic Transaction)：安全电子交易协议。电子商务，身份认证。普遍的说法是将其归为应用层。

IPsec (Internet Protocol Security)：对 IP 包加密。

29、Redis 集群切片的常见方式：

集群切片方式	核心特点
客户端分片	在客户端通过 key 的 hash 值对应到不同的服务器。
中间件实现分片	在应用软件和 Redis 中间，例如：Twemproxy、Codis 等，由中间件实现服务到后台 Redis 节点的路由分派。
客户端服务端协作分片	客户端与服务端协作完成分片处理。

30、Redis 分布式存储方案

分布式存储方案	核心特点
主从 (Master/Slave) 模式	一主多从，故障时手动切换。
哨兵 (Sentinel) 模式	有哨兵的一主多从，主节点故障自动选择新的主节点。
集群 (Cluster) 模式	分节点对等集群，分 slots，不同 slots 的信息存储到不同节点。

31、Web 应用架构设计原则

【分离关注点】分层架构背后的核心设计思想就是关注点分离。

【封装】强调应用程序各个部分应进行封装。

【依赖关系反转】依赖抽象而不是实现。

【显式依赖关系】方法和类应显式要求正常工作所需的任何协作对象。

【单一责任】对象只应有一个责任，并且只能因为一个原因更改对象。

【避免自我重复】将逻辑封装在编程构造中，而不要重复该逻辑。

【持久性无知】应用层的组件或服务不应该直接了解或关心数据的持久化机制（即数据是如何存储在数

据库或其它存储系统中的)。

【有界上下文】各 Web 应用程序应努力成为自己的有界上下文，为其业务模型提供自己的持久性存储，而不是与其它应用程序共享数据库。

32、区块链

区块链分类：

(1) 公有链 (PublicBlockChains)

无官方组织及管理机构，无中心服务器，参与的节点按照系统规则自由接入网络、不受控制，节点间基于共识机制开展工作。

公有链包括比特币、以太坊、超级账本、大多数山寨币以及智能合约，其中公有链的始祖是比特币区块链。

(2) 私有链 (PrivateBlockChains)

建立在某个企业内部，系统的运作规则根据企业要求进行设定，修改甚至是读取权限仅限于少数节点，同时仍保留着区块链的真实性和部分去中心化的特性。

(3) 联盟链 (ConsortiumBlockChains)

由若干机构联合发起，介于公有链和私有链之间，兼具部分去中心化的特性。

特点：

特点	描述
去中心化	由于使用分布式核算和存储，不存在中心化的硬件或管理机构，任意节点的权利和义务都是均等的，系统中的数据块由整个系统中具有维护功能的节点来共同维护
开放性	系统是开放的，如：区块链上的【交易信息是公开的】，不过【账户身份信息是高度加密的】
自治性	区块链采用基于协商一致的规范和协议（比如一套公开透明的算法），使得整个系统中的所有节点能够在信任的环境自由安全的交换数据，使得对“人”的信任改成了对机器的信任，任何人为的干预不起作用
安全性 (防篡改)	数据在多个节点存储了多份，篡改数据得改掉 51%节点的数据，这太难。同时，还有其它安全机制，如：比特币的每笔交易，都由付款人用私钥签名，证明确实是他同意向某人付款，其它人无法伪造
匿名性 (去信任)	由于节点之间的交换遵循固定的算法，其数据交互是无需信任的（区块链中的程序规则会自行判断活动是否有效），因此交易对手无须通过公开身份的方式让对方对自己产生信任，对信用的累积非常有帮助

区块链的核心技术：

技术	描述
分布式存储	分布式储存，简单来说，就是一种将数据分散存储到多个地方的数据储存技术，而且存储的数据可在多个参与者之间共享，人人可以参与，并具有相同的权力，一起记录数据，主要起到了数据储存的功能。
共识机制	因为区块链的分布式网络中，没有中央权威。因此，网络需要一个决策机制来促成参与者达成一致。而共识机制就是一种协调大家处理数据的机制。共识机制就决定了这些数据中，谁获得数据的记账权。
智能合约	智能合约，是一种旨在以信息化方式传播、验证或执行合同的计算机协议。智能合约可以在没有第三方的情况下，也能进行可信的交易，而且这些交易可追踪且不可逆转。所以，智能合约在系统中，主要起到了数据的执行作用。【执行智能合约的机制：去中心化网络+共识机制+受激励的矿工】
密码学	密码学，是一种特殊的加密和解密技术，区块链系统中，应用了多种多样的密码学技术，包括哈希算法、公钥私钥、数字签名等等，以此来保证整个系统的数据安全，并且证明了数据的归属。

33、Lambda 架构与 Kappa 架构对比和设计选择

对比内容	Lambda 架构	Kappa 架构
复杂度与开发、维护成本	需要维护两套系统（引擎），复杂度高，开发、维护成本高	只需要维护一套系统（引擎），复杂度低，开发、维护成本低
计算开销	需要一直运行批处理和实时计算，计算开销大	必要时进行全量计算，计算开销相对较小
实时性	满足实时性	满足实时性
历史数据处理能力	批式全量处理，吞吐量大，历史数据处理能力强	流式全量处理，吞吐量相对较低，历史数据处理能力相对较弱
使用场景	直接支持批处理，更适合对历史数据分析查询的场景，期望尽快得到分析结果，批处理可以更直接高效地满足这些需求。	不是 Lambda 的替代架构，而是简化，Kappa 放弃了对批处理的支持，更擅长业务本身为增量数据写入场景的分析需求
选择依据	◆ 根据两种架构对比分析，将业务需求、技术要求、系统复杂度、开发维护成本和历史数据处理能力作为选择考虑因素。 ◆ 计算开销虽然存在一定差别，但是相差不是很大，所以不作为考虑因素。	

34、测试阶段划分

测试阶段	内容
单元测试	依据详细设计，模块测试，模块功能、性能、接口等
集成测试	依据概要设计，模块间的接口
系统测试	依据需求文档，在真实环境下，验证完整的软件配置项能否和系统正确连接
确认测试	依据需求文档，验证软件与需求的一致性。内部确认测试、Alpha 测试、Beta 测试、验收测试

回归测试：测试软件变更之后，变更部分的正确性和对变更需求的符合性。

35、软件测试类型

35.1 以测试过程中程序执行状态为依据分类

(1) 动态测试【计算机运行程序】

分为 3 个步骤：构造测试实例、执行程序以及分析结果

(2) 静态测试【人工监测和计算机辅助分析】

静态测试的方法主要有桌前检查、代码走查、代码审查

静态测试都是做的静态分析，静态分析有：

- 控制流分析：是否存在没有使用的语句/无法达到的语句/调用并不存在的子程序。
- 数据流分析：引用未定义的变量、对以前未使用的变量再次赋值。
- 接口分析：模块之间接口的一致性、子程序和函数之间的接口一致性、函数形参与实参的数量、顺序、类型的一致性。
- 表达式分析：括号不配对、数组引用越界、除数为零。

35.2 以内部结构是否可见分类

(1) 白盒测试【结构测试】：关注内部结构与逻辑

控制流分析、数据流分析、路径分析、程序变异【错误驱动测试】

【路径覆盖】（最强）

↓
【逻辑覆盖】（由强到弱排列）
 修改条件/判定 条件组合 条件/判定覆盖 条件覆盖 判定覆盖 语句覆盖

(2) 黑盒测试【功能测试】：关注输入输出及功能

等价类划分：不同等价类，揭示不同问题；有效等价类/无效等价类。

边界值分析：1≤x≤10，可取 x 的值为 0、1、10 和 11 作为测试数据。

错误推测：依靠测试人员的经验和直觉。

判定表：最适合描述在多个逻辑条件取值的组合所构成的复杂情况下，分别要执行哪些不同的动作。

因果图：根据输入条件与输出结果之间的因果关系来设计测试用例。

(3) 灰盒测试法【既关注逻辑又关注功能】

35.3 从程序执行的方式分类

(1) 人工测试

(2) 自动化测试

先写脚本→自动化执行

不适合场景：项目周期短，需求变动频繁

常见自动化测试：单元自动化测试、接口自动化测试、UI 自动化测试

36、主从数据库

(1) 主从数据库结构特点：

一般：一主多从，也可以多主多从。

主库做写操作，从库做读操作。

(2) 主从复制步骤：

主库 (Master) 更新数据完成前，将操作写 binlog 日志文件。

从库 (Slave) 打开 I/O 线程与主库连接，做 binlog dump process，并将事件写入中继日志。

从库执行中继日志事件，保持与主库一致。

37、微服务的概念、优势及其面临的挑战

微服务顾名思义，就是很小的服务，所以它属于面向服务架构的一种。微服务在更小的基础上，其实进一步在突显其独立性。

微服务的优势：小服务【且专注于做一件事情】；轻量级的通信机制；松耦合；独立测试；独立部署【简单部署】；独立运行【每个服务独立在其独立进程中】；支持异构【如每个服务使用不同数据库】；更好的可用性与弹性；化整为零，易于小团队开发。

微服务面临的挑战：分布式环境下的数据一致性【更复杂】；测试的复杂性【服务间依赖测试】；运维的复杂性。

38、知识产权人确定

情况说明		判断说明	归属
作品	职务作品	利用单位的物质技术条件进行创作，并由单位承担责任的	除署名权外其它著作权归单位

		有合同约定，其著作权属于单位	除署名权外其它著作权归单位
		其它	作者拥有著作权，单位有权在业务范围内优先使用
软件	职务作品	属于本职工作中明确规定的开发目标	单位享有著作权
		属于从事本职工作活动的结果	单位享有著作权
		使用了单位资金、专用设备、未公开的信息等物质、技术条件，并由单位或组织承担责任的软件	单位享有著作权
专利权	职务作品	本职工作中做出的发明创造	单位享有专利权
		履行本单位交付的本职工作之外的任务所做出的发明创造	单位享有专利权
		离职、退休或调动工作后 1 年内，与原单位工作相关	单位享有专利权
作品 软件	委托创作	有合同约定，著作权归委托方	委托方
		合同中未约定著作权归属	创作方
	合作开发	只进行组织、提供咨询意见、物质条件或者进行其它辅助工作	不享有著作权
		共同创作的	共同享有，按人头比例。 成果可分割的，可分开申请。
商标		谁先申请谁拥有（除知名商标的非法抢注） 同时申请，则根据谁先使用（需提供证据） 无法提供证据，协商归属，无效时使用抽签（但不可不确定）	
专利		谁先申请谁拥有 同时申请则协商归属，协商不成则同时驳回双方的专利申请	

39、五大架构风格

五大架构风格	子风格	核心特点及注意事项
数据流风格 Data Flow	批处理 Batch Sequential 管道-过滤器 Pipes and Filters	数据驱动, 适合用于前一步处理结果作为后一步数据处理输入的场景。
调用/返回风格	主程序/子程序	层次型就属于这种, 通过分层做

Call/Return	Main Program and Subroutine 面向对象 Object-oriented 分层架构 Layered System	内容的解耦。如：网络七层模型，B/S。
独立构件风格 Independent Components	进程通信 Communicating Processes 事件驱动系统（隐式调用） Event system	强调不直接调用，事件触发是典型。如：订阅-发布模式。
虚拟机风格 Virtual Machine	解释器 interpreter 规则系统 Rule-based System	用于自定义场景。能不写代码自定义业务流程并执行。如：解释器。
以数据为中心 Data-centered	数据库系统 Database System 黑板系统 Blackboard System 超文本系统 Hypertext System	各种构件围绕【数据中心】工作。如：数据库系统、黑板风格。

40、CPS 的自动控制系统行为模式

(1) 开环控制



(2) 闭环控制



(3) 顺序控制：即按照固定的顺序逐步执行的过程，或按照既定的逻辑有序执行的过程，如基于系统状态和输入而执行不同行为的过程。顺序控制可看作开环控制和闭环控制的扩展，它将若干开环或闭环过程串联起来，整个顺序控制过程通常产生若干输出实例。

(4) 批量控制：即批处理过程，是指利用一个或以上设备，对输入的有限数量的材料按照事先预定的处理行为进行加工的控制过程。

41、CPS 分层架构

(1) 单元级

特点：不可分割性；硬件与软件的集成；数据闭环。

功能：可感知；可计算；可交互；可延展；自决策。

(2) 系统级

特点：“一硬、一软、一网”的有机组合；数据流动与资源共享；互联、互通与互操作。

功能：实现多个单元级 CPS 的互联互通和互操作，进一步提高制造资源优化配置的广度、深度和精度。

(3) 系统之系统级 (SoS)

多个系统级 CPS 的有机组合，涵盖“一硬、一软、一网、一平台”四大核心技术要素。

42、容器化和自动化部署工具

Docker	轻量级虚拟化容器技术、基于 Linux 内核。 【轻量级】秒级启停，高度可移植性和灵活性 【快速部署和运行】通过打包可移植镜像达成 【隔离性】每个容器有自己的命名空间、文件系统和网络资源 【可扩展性】很容易做横向和纵向扩展 【易于管理】有一套易于使用的命令行工具和 API 接口
Kubernetes (K8s)	开源的容器编排平台（自动化部署、扩展和管理容器化应用程序） 【Master】管理整个集群的状态，决策集群上的 Node 如何部署容器 【Node】工作节点，负责运行和管理容器 【Pod】最小调度单元，可包含 1 个或多个容器，共享命名空间和存储 【Service】用于定义一组 Pod 的访问入口 【Volume】为 Pod 提供持久化存储 【NameSpace】将集群划分为多个虚拟集群，让团队可独立管理自己的应用
Jenkins	自动化构建和持续集成工具 Docker、Kubernetes、GitLab 等都可充当 Jenkins 的插件

43、服务注册和发现工具

Consul	分布式的服务注册与发现工具。 主要功能： 【服务注册】服务注册包括：服务名称+服务地址+端口号。 【服务发现】客户端通过 Consul 查询 API 可用服务并返回健康的服务列表。通过 DNS
--------	---

	或 HTTP 等多种方式实现。 【健康检查】定期检查，对不可用服务标记从可用列表中删除。 【多数据中心】多数据中心带来更好的可用性和容错性。 【KV 存储】用于存储配置数据和共享数据。
ZooKeeper	分布式的开源协调服务，用于处理分布式应用中的协作问题（分布式锁、选举、配置管理等） 特点：高可用、高性能和严格一致性 ZooKeeper 数据存储在内存中 ZooKeeper 中数据结构称为 Znode ZooKeeper 通常用于服务注册和服务发现
Eureka	基于 REST 的服务治理框架，用于服务注册和发现 Eureka Server：服务注册中心 Eureka Client：服务提供方，将自己的服务注册到 Server，并从 Server 获取其它服务注册信息
Etcd	分布式的、高可用的键值存储系统，主要进行元数据管理和配置共享 已成为 Kubernetes 的默认数据存储和服务发现组件 特点： 【高可用性】用 Raft 协议来保证数据的一致性和高可用性 【快速响应】Go 语言开发，具有高效的性能和快速响应能力 【可靠性】强大的事务机制 【灵活性】支持监听，可及时捕捉数据变化，支持多种数据格式

44、REST

(1) 概念：REST (Representational State Transfer，表述性状态转移) 是一种通常使用 HTTP 和 XML 进行基于 Web 通信的技术，可以降低开发的复杂性，提高系统的可伸缩性。

(2) REST 相关概念：

资源：如架构课程订单

表述：资源某一时间的状态，呈现形式有 HTML、JSON、XML

状态转移：使用 GET、POST 等方法

超链接：通过超链接可与其它资源联系

(3) REST 的 5 个原则

网络上的所有事物都被抽象为资源。

每个资源对应一个唯一的资源标识（反过来不成立）。

通过通用的连接件接口对资源进行操作。

对资源的各种操作不会改变资源标识。

所有的操作都是无状态的。

(4) REST API

命令	用途
GET	从指定的资源请求数据，只进行数据检索，不进行其他操作。
POST	将数据发送到服务器进行创建，通常用于上传文件或提交表单。
PUT	更新目标资源的所有当前表示，使用上传的内容进行替换。
DELETE	删除指定的资源。
HEAD	与 GET 方法类似，但只传输状态行和头部信息。
PATCH	对资源进行部分修改。

45、微服务与 SOA 的对比

微服务	SOA
能拆分的就拆分	是整体的，服务能放一起的都放一起
纵向业务划分	是水平分多层
由单一组织负责	按层级划分不同部门的组织负责
细粒度	粗粒度
两句话可以解释明白	几百字只相当于 SOA 的目录
独立的子公司	类似大公司里面划分了一些业务单元（BU）
组件小	存在较复杂的组件
业务逻辑存在于每一个服务中	业务逻辑横跨多个业务领域
使用轻量级的通信方式，如 HTTP	企业服务总线（ESB）充当了服务之间通信的角色

46、移动端应用部署平台

(1) 小程序部署平台

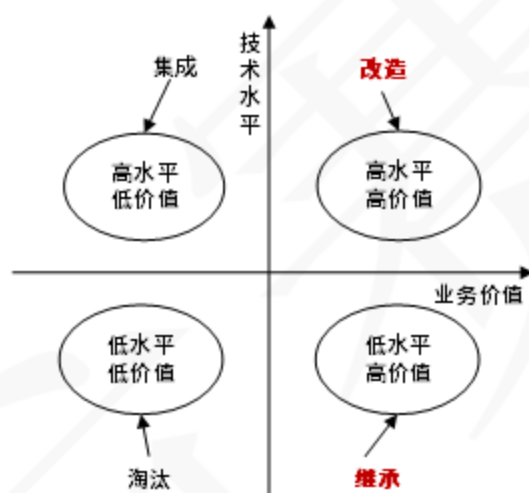
常见的移动端应用部署平台包括微信小程序、QQ 小程序、抖音小程序、支付宝小程序等。

(2) App 部署平台

主流的 App 部署平台——Android 系统和 iOS 系统自带的官方应用商店（如 Google Play 和 App Store）。

比较维度	iOS	Android
开源和闭源	闭源	开源
硬件资源的使用效率不同	运行效率高	使用效率低
对应的扩展程序优化不同	需要优化到位	多数无法优化到位
运行机制	沙盒运行机制	虚拟机运行机制
后台程序运行	自动关闭后台，释放空间	不会自动关闭后台，占用内存
屏幕响应优先级	最先响应屏幕	第三层级响应
系统开放性	严格的审核机制	缺乏监管

47、遗留系统演化策略



淘汰策略：遗留系统的技术含量较低，且具有较低的业务价值。对遗留系统的完全淘汰是企业资源的根本浪费，系统分析师应该善于“变废为宝”，通过对遗留系统功能的理解和借鉴，可以帮助新系统的设计，降低新系统开发的风险。

继承策略：遗留系统的技术含量较低，已经满足企业运作的功能或性能要求，但具有较高的商业价值，目前企业的业务尚紧密依赖该系统。对这种遗留系统的演化策略为继承。在开发新系统时，需要完全兼容遗留系统的功能模型和数据模型。为了保证业务的连续性，新老系统必须并行运行一段时间，再逐渐切换到新系统上运行。

改造策略：遗留系统具有较高的业务价值，基本上能够满足企业业务运作和决策支持的需要。这种系统可能建成的时间还很短，对这种遗留系统的演化策略为改造。改造包括系统功能的增强和数据模型的改造两个方面。系统功能的增强是指在原有系统的基础上增加新的应用要求，对遗留系统本身不做改变；数据模型的改造是指将遗留系统的旧的数据模型向新的数据模型的转化。

集成策略：遗留系统的技术含量较高，但其业务价值较低，可能只完成某个部门（或子公司）的业务管理。这种系统在各自的局部领域里工作良好，但对于整个企业来说，存在多个这样的系统，不同的系统基于不同的平台、不同的数据模型，形成了一个个信息孤岛，对这种遗留系统的演化策略为集成。

48、I/O 控制方式

（1）程序控制（查询）方式：分为无条件传送和程序查询方式两种。方法简单，硬件开销小，但 I/O 能力不高，严重影响 CPU 的利用率。

（2）程序中断方式：与程序控制方式相比，中断方式因为 CPU 无需等待而提高了传输请求的响应速度。【CPU 与 I/O 传输可并行】

（3）DMA 方式：DMA 方式是为了在主存与外设之间实现高速、批量数据交换而设置的。DMA 方式比程序控制方式与中断方式都高效。【CPU 与 I/O 传输可并行】

（4）通道方式

（5）I/O 处理机

49、嵌入式实时操作系统调度算法

（1）基于优先级的抢占调度：基于优先级的抢占调度在任何时候运行的任务都是所有就绪任务中具有最高优先级的任务。优先级可以分为静态优先级和动态优先级。

（2）时间轮转调度算法：为每个任务提供确定份额的 CPU 执行时间。

（3）单调速率调度（Rate Monotonic Scheduling, RMS）算法：静态固定优先级算法，任务周期越短优先级越高，是静态调度中最优的。

（4）截止时间优先（Earliest Deadline First, EDF）调度算法（最早截止期调度算法）：根据任务的截止时间来确定其优先级，对于时间期限最近的任务，分配最高的优先级。在动态调度算法中 CPU 的利用率达到了最高。

（5）最低松弛度优先（Least Laxity First, LLF）算法：根据任务紧急（或松弛）的程度，来确定任务的优先级。任务的紧急程度愈高，为该任务所赋予的优先级就愈高，使之优先执行。动态调度算法。

（6）基于分区（partition）化的分区 APPS 调度算法：将处理器时间资源划分成多个独立时间片资源，以供多个分区实体独立享用。通过遵循特定的分区调度原则和时间优先调度（TPS）策略，该算法可以确保系统的稳定性和性能。静态调度算法。

50、操作系统内核架构的比较

（1）内核是操作系统的核心部分，它管理着系统的各种资源。内核可以看成连接应用程序和硬件的一座桥梁，是直接运行在硬件上的最基础的软件实体。目前从内核架构来划分，可分为宏内核（Monolithic

Kernel) 和微内核 (Micro Kernel)

(2) 操作系统内核架构的比较

	实质	优点	缺点
宏内核	高度集成。用户服务和内核服务在同一空间中实现。	实时性高；结构比较简单，易于设计。	代码耦合度高，缺少移植性。
微内核 【鸿蒙操作系统】	模块化。用户服务和内核服务不在同一空间中实现。	结构清晰，有利于协作开发；有良好的移植性，代码量非常少；有相当好的伸缩性、扩展性；可用于分布式系统。	系统性能偏低。

51、大数据

(1) 概念

大数据是具有数量巨大、来源多样、生成极快、多变等特征且难以使用传统数据体系结构有效处理的包含大量数据集的数据

(2) 大数据的 5 个 V:

大规模 (Volume)

高速度 (Velocity)

多样化 (Variety)

价值密度低 (Value)

真实性 (Veracity)

(3) 通用设计原则

可扩展 (分布式、模块化)

可管理 (管理策略和方法)

数据安全 (数据加密、访问控制、数据备份与恢复、审计日志、持续监控)

高性能 (数据分区、分布式计算、并行处理、内存计算)

高可用 (数据冗余、自动故障转移、负载均衡)

稳定性 (监控和预警、性能调优、故障排除、升级和维护)

(4) 大数据面临着 5 个主要问题

异构性 (Heterogeneity)、规模 (Scale)、时间性 (Timeliness)、复杂性 (Complexity) 和隐私性 (Privacy)

52、嵌入式微处理器体系结构

体系结构分类	定义	特点	典型应用
冯·诺依曼结构 (普林斯顿结构)	是一种将程序指令存储器和数据存储器合并在一起的存储器结构。	指令与数据存储器合并在一起。 指令与数据都通过相同的数据总线传输。	一般用于 PC 处理器,如 I3、I5、I7 处理器。 注: 常规计算机属于冯·诺依曼结构。
哈佛结构	是一种并行体系结构,它的主要特点是将程序和数据存储在不同的存储空间中,即程序存储器和数据存储器是两个独立的存储器,每个存储器独立编址、独立访问。	指令与数据分开存储,可以并行读取,有较高的数据吞吐率。 有 4 条总线: 指令和数据的数据总线与地址总线。	一般用于嵌入式系统处理器。 注: DSP 属于哈佛结构。

53、云计算

(1) 概念

云计算是集合了大量计算设备和资源,对用户屏蔽底层差异的分布式处理架构,其用户与提供实际服务的计算资源是相分离的。

(2) 优点

超大规模、虚拟化、高可靠性、高可伸缩性、按需服务、成本低(前期投入低、综合使用成本也低)

(3) 分类

按服务类型分类:

软件即服务(SaaS): 基于多租户技术实现,直接提供应用程序。

平台即服务(PaaS): 虚拟中间件服务器、运行环境和操作系统。

基础设施即服务(IaaS): 包括服务器、存储和网络等服务。

按部署方式分类:

公有云: 面向互联网用户需求,通过开放网络提供云计算服务。

社区云: 云基础设施分配给一些社区组织所专有。

私有云: 面向企业内部提供云计算服务。

混合云: 兼顾以上两种情况的云计算服务。

54、设计模式

(1) 创建型模式：创建型模式与对象的创建有关，抽象了实例化过程，它们帮助一个系统独立于如何创建、组合和表示它的那些对象。一个类创建型模式使用继承改变被实例化的类，而一个对象创建型模式将实例化委托给另一个对象。

设计模式名称	简要说明	速记关键字
Factory Method 工厂方法模式	定义了创建对象的接口，它允许子类决定实例化哪个类	动态生产对象
Abstract Factory 抽象工厂模式	提供一个接口，可以创建一系列相关或相互依赖的对象，而无需指定它们具体的类	生产成系列对象
Builder 构建器模式	将一个复杂对象的表示与其构造相分离，使得相同的构建过程能够得出不同的表示	复杂对象构造
Prototype 原型模式	允许对象在不了解要创建对象的确切类以及如何创建等细节的情况下创建自定义对象。通过拷贝原型对象来创建新的对象	克隆对象
Singleton 单例模式	确保一个类只有一个实例，并且提供了对该类的全局访问入口	单实例

(2) 行为型模式：行为型模式涉及算法和对象间职责的分配。行为模式不仅描述对象或类的模式，还描述它们之间的通信模式。行为型类模式使用继承机制在类间分配行为，这里包括模板类模式和解释器类模式。行为对象模式使用对象复合而不是继承。一些行为对象模式描述了一组对等的对象怎样相互协作以完成其中任一对象都无法单独完成的任务。

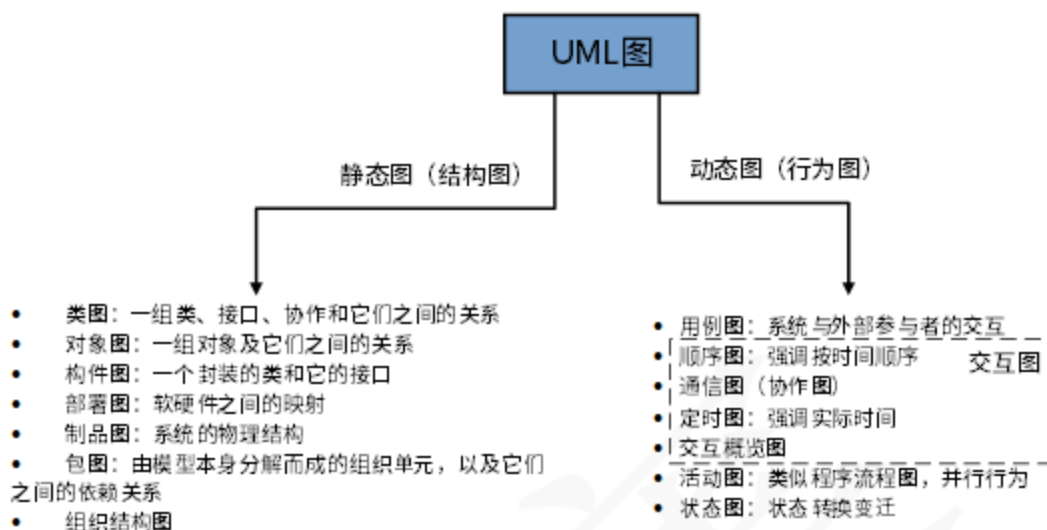
设计模式名称	简要说明	速记关键字
Chain of Responsibility 职责链模式	可以在系统中建立一个链，这样消息可以在首先接收到它的级别处被处理，或者可以定位到可以处理它的对象	传递职责
Command 命令模式	在对象中封装了请求，这样就可以保存命令，将该命令传递给方法以及像任何其他对象一样返回该命令	日志记录，可撤销
Interpreter 解释器模式	可以解释定义其语法表示的语言，还提供了用表示来解释语言中的语句的解释器	虚拟机的机制
Iterator 迭代器模式	为集合中的有序访问提供了一致的方法，而该集合是独立于基础集合，并与之相分离的	数据集
Mediator 中介者模式	通过引入一个能够管理对象间消息分布的对象，简化了系统中对象间的通信	不直接引用

Memento 备忘录模式	保持对象状态的“快照”（snapshot），这样对象可以在不向外界公开其内容的情况下返回到它的最初状态	游戏存档
Observer 观察者模式	为组件向相关接收方广播消息提供了灵活的方法 定义对象间的一种一对多的依赖关系	订阅、广播、联动
State 状态模式	允许一个对象在其内部状态改变时改变它的行为	状态变成类
Strategy 策略模式	定义一系列算法，把它们一个个封装起来，并且使它们之间可互相替换，从而让算法可以独立于使用它的用户而变化	多方案切换
Template Method 模板方法模式	提供了在不重写方法的前提下允许子类重载部分方法的方法	框架
Visitor 访问者模式	提供了一种方便的、可维护的方法来表示在对象结构元素上要进行的操作	数据与操作分离

（3）结构型模式：结构型模式处理类或对象的组合，结构型设计模式涉及如何组合类和对象以获得更大的结构。结构型类模式采用继承机制来组合接口或实现。结构型对象模式不是对接口和实现进行组合，而是描述了如何对一些对象进行组合，从而实现新功能的一些方法。

设计模式名称	简要说明	速记关键字
Adapter 适配器模式	将一个类的接口转换成用户希望得到的另一种接口。它使原本不相容的接口得以协同工作	转换接口
Bridge 桥接模式	将一个复杂的组件分成两个独立的但又相关的继承层次结构 将类的抽象部分和它的实现部分分离开来，使它们可以独立地变化	继承树拆分
Composite 组合模式	创建树型层次结构来改变复杂性，同时允许结构中的每一个元素操作同一个接口。用于表示“整体-部分”的层次结构	树形目录结构
Decorator 装饰模式	在不修改对象外观和功能的情况下添加或者删除对象功能 即动态地给一个对象添加一些额外的职责	动态附加职责
Facade 外观模式	子系统中的一组接口提供了一个统一的接口	对外统一接口
Flyweight 享元模式	可以通过共享对象减少系统中低等级的、详细的对象数目 提供支持大量细粒度对象共享的有效方法	汉字编码
Proxy 代理模式	为控制对初始对象的访问提供了一个代理或者占位符对象	快捷方式

55、UML 图



56、数据加密技术

加密机制有助于保护信息的机密性和完整性，有助于识别信息的来源，它是最广泛使用的安全机制。

56.1 对称加密

- （1）对称加密（又称为私人密钥加密/共享密钥加密）：加密与解密使用同一密钥。
- （2）特点：加密强度不高，但效率高，易破解；密钥分发困难。
- （3）用途：对消息明文进行加密传送
- （4）常见对称密钥加密算法：

DES：替换+移位、56 位密钥、64 位数据块、速度快、密钥易产生

3DES（三重 DES）：密钥长度 112【两个 56 位的密钥 K1、K2】

加密：K1 加密->K2 解密->K1 加密

解密：K1 解密->K2 加密->K1 解密

IDEA：128 位密钥、64 位明文/密文，PGP 采用该算法。

其它：RC-5、AES。

56.2 非对称加密

- （1）非对称加密（又称为公开密钥加密）：密钥必须成对使用（公钥加密，相应的私钥解密）。
- （2）特点：加密强度高，但效率低，极难破解；密钥分发容易。
- （3）用途：对密钥加密，做数字签名
- （4）常见非对称密钥加密算法：

RSA：官方教程上是 512 位密钥，不过现在已经支持 2048 位（或 1024 位）密钥。

Elgamal：安全性依赖于计算有限域上离散对数这一难题

ECC：椭圆曲线算法

57、业务流程分析方法

价值链分析法：找出或设计出那些能够使顾客满意，实现顾客价值最大化的业务流程。

客户关系分析法：把 CRM 用在业务流程的分析上。

供应链分析法：从企业供应链的角度分析企业的业务流程。

基于 ERP 的分析法：将企业的业务流程看作是一个紧密联接的供应链，将供应商和企业内部的采购、生产、销售，以及客户紧密联系起来，对供应链上的所有环节进行有效管理，实现对企业的动态控制和各种资源的集成和优化。

业务流程重组：通过重新审视企业的价值链，从功能成本的比较分析中，确定企业在哪些环节具有比较优势。

58、数据流图

数据流图绘制基本原则：

- (1) DFD 中只用“外部实体、数据流、加工、存储”这四种图形元素，每个元素都有名字。
- (2) 每个加工至少有一个输入数据流和一个输出数据流，而且要保持【数据守恒】。
- (3) 在 DFD 中，按层给加工编号（编号表明了该加工处在哪一层，以及上下层的父图与子图的对应关系）。
- (4) 规定任何一个 DFD 子图必须与它上一层的一个加工对应，两者的输入数据流和输出数据流必须一致。此即父图与子图的平衡。
- (5) 在整套 DFD 中，每个数据存储必须既有读的数据流，又有写的数据流。但是在某张子图中，可能只有读没有写，或者只有写没有读。
- (6) 可以在 DFD 中加入物质流，帮助用户理解 DFD，但不可夹带控制流。

高质量数据流图绘制原则：

- (1) 复杂性最小化原则。数据流图分层结构就是把信息划分为小的且相对独立的一大批子集例子，这样就可以单独考察每一个数据流图。如果要了解某个过程更加详细的信息，可以跳转到该过程的下一层；如果要知道一个数据流图如何与其它数据流图相关联，可以跳转到上一层的数据流图进行考察。
- (2) 接口最小化原则。接口最小化是复杂性最小化的一种具体规则。在设计模式时，应使得模型中各个元素之间的接口数或连接数最小化。
- (3) 数据流一致性原则。一个过程和它的过程分解在数据流内容中是否有差别？是否存在有数据流出但没有相应的数据流入的加工？是否存在有数据流入但没有相应的数据流出的加工？

数据流图的优缺点

优 点	缺 点
易于理解和使用，可以帮助系统分析师和软件设计师更好地理解和描述系统功能	只适用于描述系统功能，不能很好地描述系统的数据结构和算法
可以帮助设计人员更好地分析和设计系统，优化系统结构和流程，提高系统性能和效率	不够灵活，难以应对复杂系统的需求和变化
可以用于不同层次的系统分析和设计，从概念层到实现层都可以使用数据流图进行描述	不能提供系统实现的详细信息，需要结合其他工具和技术进行完整的系统设计和实现
可以与其他软件工程工具和技术相集成，如需求分析工具、结构化设计工具、面向对象设计工具等	

制作于 26 年 4 月 适用于第 2 版教材